SOFTWARE ENGINEERING · CHAPTER 04

Lab-04: RESTful Web Services (JAX-RS)

You build **shop-rest**, a product catalogue API with Jakarta REST 4.0 on Eclipse Jersey and an embedded Grizzly server. Task 1 starts the server and serves the first resource; Task 2 completes the CRUD with correct status codes, headers and JSON/XML representations; Task 3 adds typed parameters, a `ParamConverter`, Bean Validation and problem+json errors; Task 4 adds filters and interceptors (correlation id, CORS, API key, gzip); Task 5 adds pagination with `Link` headers, ETag caching, a v2 media type and a sub-resource. The challenge adds an orders sub-API with HATEOAS links, idempotent POSTs, an OpenAPI document and a full `JerseyTest` suite.

Jakarta REST 4.0 · Jersey 4 · Grizzly | JDK 25 · Maven 3.9 | ≈ 5 hours | 5 tasks + 1 challenge

How to work through this lab. Do the tasks in order; each builds on the previous. Read the *Goal*, follow the *Steps*, compare with the *Expected output*, and tick the *Acceptance checklist* (saved in your browser). Open a hint only when stuck for more than 10 minutes. Keep a terminal with `curl` open: every task ends with a recorded HTTP transcript in `docs/`.

0 Setup — 15 min

1. Verify the tooling. You need JDK 25, Maven 3.9.16 and curl (bundled with Windows 10+, macOS and most Linux distributions).

```
java -version      # openjdk 25
mvn -version       # Apache Maven 3.9.16
curl --version
```

2. Create the Maven project `shop-rest` with this `pom.xml`. The Jersey BOM pins every Jersey module; the JUnit BOM pins JUnit 6.1.3 for the challenge.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>edu.itc</groupId>
  <artifactId>shop-rest</artifactId>
```

```
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.release>25</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <jersey.version>4.0.0</jersey.version>      <!-- use the latest 4.0.x on
Maven Central -->
    <junit.version>6.1.3</junit.version>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.glassfish.jersey</groupId>
        <artifactId>jersey-bom</artifactId>
        <version>${jersey.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
      <dependency>
        <groupId>org.junit</groupId>
        <artifactId>junit-bom</artifactId>
        <version>${junit.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>

  <dependencies>
    <!-- Jakarta REST 4.0 API arrives transitively (jakarta.ws.rs-api) -->
    <dependency>
      <groupId>org.glassfish.jersey.containers</groupId>
      <artifactId>jersey-container-grizzly2-http</artifactId>
    </dependency>
    <dependency>
      <groupId>org.glassfish.jersey.inject</groupId>
      <artifactId>jersey-hk2</artifactId>
    </dependency>
    <dependency>
      <groupId>org.glassfish.jersey.media</groupId>
      <artifactId>jersey-media-json-binding</artifactId>      <!-- JSON-B
(Yasson) -->
    </dependency>
    <dependency>
      <groupId>org.glassfish.jersey.media</groupId>
      <artifactId>jersey-media-jaxb</artifactId>              <!-- XML -->
    </dependency>
```

```
        <dependency>
          <groupId>org.glassfish.jaxb</groupId>
          <artifactId>jaxb-runtime</artifactId>
          <version>4.0.5</version>
        </dependency>
        <dependency>
          <groupId>org.glassfish.jersey.ext</groupId>
          <artifactId>jersey-bean-validation</artifactId>         <!-- Hibernate
    Validator -->
        </dependency>

        <dependency>
          <groupId>org.junit.jupiter</groupId>
          <artifactId>junit-jupiter</artifactId>
          <scope>test</scope>
        </dependency>
        <dependency>
          <groupId>org.glassfish.jersey.test-framework.providers</groupId>
          <artifactId>jersey-test-framework-provider-grizzly2</artifactId>
          <scope>test</scope>
        </dependency>
      </dependencies>

      <build>
        <plugins>
          <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.5.3</version>
          </plugin>
          <plugin>
            <groupId>org.codehaus.mojo</groupId>
            <artifactId>exec-maven-plugin</artifactId>
            <version>3.5.0</version>
            <configuration><mainClass>edu.itc.shop.Main</mainClass>
    </configuration>
          </plugin>
        </plugins>
      </build>
    </project>
```

3. Create the package layout. Jersey scans `edu.itc.shop.api` (and its sub-packages) for `@Path` and `@Provider` classes.

```
shop-rest/
├── pom.xml
├── README.md
├── docs/                          # curl transcripts, one file per task
```

```
└── src/
    ├── main/java/edu/itc/shop/
    │   ├── Main.java              # Grizzly bootstrap + ResourceConfig
    │   ├── domain/                # Product, Money, repositories (no HTTP
here)
    │   └── api/
    │       ├── ProductResource.java
    │       ├── dto/               # ProductDto, StockChange, Problem, ...
    │       ├── params/            # ProductFilter, MoneyConverterProvider
    │       ├── error/             # exception mappers
    │       └── filter/            # filters and interceptors
    └── test/java/edu/itc/shop/    # JerseyTest classes (challenge)
```

4. Run `mvn -q compile` once so all dependencies are downloaded. The server is started later with `mvn -q compile exec:java` and stopped with `Ctrl` + `C` .

1 First resource and server

Easy — 30 min

GOAL

Start Jersey on an embedded Grizzly server at `http://localhost:8080/api/` and serve `GET /products` and `GET /products/{id}` from an in-memory repository, with a real 404 for unknown ids.

STEPS

1. In `domain` create the record `Product(long id, String name, String sku, BigDecimal price, int stock, int version)` and the interface `ProductRepository` with `findAll()`, `findById(long)`, `findBySku(String)`, `save(Product)` (assigns an id when `id == 0`, increments `version` on every save) and `delete(long)`.
2. Implement `InMemoryProductRepository` with a `ConcurrentHashMap<Long, Product>` and an `AtomicLong` for ids (Chapter 01 rules: it is shared by all request threads). Seed three products in the constructor: Kettle `KT-01` 24.90 (stock 12), Mug `MG-01` 6.50 (stock 40), Tea `TE-01` 4.20 (stock 0).
3. Create `Main` (starter below). The static `config()` method builds the `ResourceConfig`: package scanning plus an `AbstractBinder` that binds one repository instance to `ProductRepository`. Tests reuse `config()` later.
4. Create `api/ProductResource` at `@Path("/products")` with constructor injection of `ProductRepository` (`@Inject`). Implement `GET` (list) and `GET {id}` returning the `Product` record; throw `NotFoundException` when the id is unknown. Constrain the template to digits: `@Path("{id: \\d+}")`.

5. Start the server with `mvn -q compile exec:java` and call the three URLs below with `curl -i`. Note that `/products/abc` is a 404 too (no template matches), while `/products/99` is your explicit 404.
6. Save the transcript as `docs/task1-http.md`. Write one sentence in `README.md` explaining why the resource class has no mutable fields even though it is stateless per request.

STARTER CODE

```
package edu.itc.shop;

import java.net.URI;
import org.glassfish.grizzly.http.server.HttpServer;
import org.glassfish.jersey.grizzly2.httpserver.GrizzlyHttpServerFactory;
import org.glassfish.jersey.internal.inject.AbstractBinder;
import org.glassfish.jersey.server.ResourceConfig;
import edu.itc.shop.domain.InMemoryProductRepository;
import edu.itc.shop.domain.ProductRepository;

public class Main {
    public static final URI BASE = URI.create("http://localhost:8080/api/");

    /** The application: scanned resources/providers + one shared repository. */
    public static ResourceConfig config() {
        return new ResourceConfig()
                .packages("edu.itc.shop.api")
                .register(new AbstractBinder() {
                    @Override
                    protected void configure() {
                        bind(new
InMemoryProductRepository()).to(ProductRepository.class);
                    }
                });
    }

    public static void main(String[] args) throws Exception {
        HttpServer server = GrizzlyHttpServerFactory.createHttpServer(BASE,
config());
        Runtime.getRuntime().addShutdownHook(new Thread(server::shutdownNow));
        System.out.println("shop-rest listening on " + BASE);
        Thread.currentThread().join();
    }
}


package edu.itc.shop.api;

import java.util.List;
```

```
import jakarta.inject.Inject;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import edu.itc.shop.domain.Product;
import edu.itc.shop.domain.ProductRepository;

@Path("/products")
@Produces(MediaType.APPLICATION_JSON)
public class ProductResource {
    private final ProductRepository repo;

    @Inject
    public ProductResource(ProductRepository repo) { this.repo = repo; }

    @GET
    public List<Product> list() {
        return repo.findAll();
    }

    // TODO: GET {id: \\d+} → Product or NotFoundException("Product " + id + "
not found")
}
```

EXPECTED OUTPUT

```
$ mvn -q compile exec:java
shop-rest listening on http://localhost:8080/api/

$ curl -i localhost:8080/api/products
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 219

[{"id":1,"name":"Kettle","price":24.90,"sku":"KT-01","stock":12,"version":1},
 {"id":2,"name":"Mug","price":6.50,"sku":"MG-01","stock":40,"version":1},
 {"id":3,"name":"Tea","price":4.20,"sku":"TE-01","stock":0,"version":1}]

$ curl -i localhost:8080/api/products/2
HTTP/1.1 200 OK
Content-Type: application/json

{"id":2,"name":"Mug","price":6.50,"sku":"MG-01","stock":40,"version":1}

$ curl -i localhost:8080/api/products/99
HTTP/1.1 404 Not Found
Content-Length: 0
```

```
$ curl -i localhost:8080/api/products/abc
HTTP/1.1 404 Not Found
```

Show hints

- JSON-B 3 serialises records without any annotation; properties come out in alphabetical order, which is why `version` is visible. Task 2 hides it behind a DTO.
- `AbstractBinder` lives in `org.glassfish.jersey.internal.inject` (module `jersey-common`, already on the classpath). Without the binder Jersey cannot construct `ProductResource` and answers 500.
- Use `save(new Product(0, "Kettle", "KT-01", new BigDecimal("24.90"), 12, 0))` in the seed; `save` replaces id 0 by `ids.incrementAndGet()` and stores `version + 1`.
- If port 8080 is busy (a Spring app from Lab-02 still running?), change `BASE`; every transcript must then use your port.
- Windows PowerShell aliases `curl` to `Invoke-WebRequest`; call `curl.exe` explicitly, or use Git Bash.

ACCEPTANCE CHECKLIST

- [ ] `mvn -q compile exec:java` starts Grizzly on 8080 and `GET /api/products` returns the three seeded products as JSON.
- [ ] `GET /api/products/2` returns 200 with one product; `/products/99` and `/products/abc` return 404.
- [ ] The repository is a single instance bound through `AbstractBinder` and injected into the resource constructor; the resource has no mutable fields.
- [ ] `docs/task1-http.md` contains the four curl calls with full response headers.

2 Full CRUD with status codes, headers, JSON and XML

Easy

45 min

GOAL

Complete the product resource with `POST`, `PUT`, `PATCH` and `DELETE`, answer with the correct status codes and headers (201 + `Location`, 204, 404, 409), and offer both JSON and XML representations through a DTO.

STEPS

1. Create `api/dto/ProductDto` as a JAXB-annotated class (starter below) with a static `from(Product)` and `toProduct(long id, int version)`. Replace `Product` by `ProductDto`

in every response of `ProductResource`; `version` no longer appears in the JSON.

2. Annotate the class with `@Produces({APPLICATION_JSON, APPLICATION_XML})` and `@Consumes({APPLICATION_JSON, APPLICATION_XML})`. Verify that `Accept: application/xml` yields `<product id="1">…`.
3. `POST /products`: body is a `ProductDto` (id ignored). If `findBySku` finds the SKU, throw `DuplicateSkuException` (in `domain`, extends `ClientErrorException` with status 409 for now). Otherwise save and return `201` with `Location` built from `UriInfo` and the created DTO as body.
4. `PUT /products/{id}`: replace name, sku, price and stock; 200 with the new representation, 404 when missing. `DELETE /products/{id}`: 204 on success, 404 when missing.
5. `PATCH /products/{id}/stock` with body `{"delta": -5}` (record `api/dto/StockChange(int delta)`, JSON only): 200 with the updated product; 409 (`InsufficientStockException`) when the stock would go negative.
6. Record the transcript in `docs/task2-http.md`: create (201), duplicate SKU (409), XML round trip (POST XML, GET XML), PATCH success and 409, DELETE 204 then 404, plus `Content-Type: text/plain` on POST (415, done by Jersey).

STARTER CODE

```
package edu.itc.shop.api.dto;

import java.math.BigDecimal;
import jakarta.xml.bind.annotation.*;
import edu.itc.shop.domain.Product;

// One class serves JSON (JSON-B) and XML (JAXB). JAXB needs a class with a
// no-arg constructor; records are JSON-only, which is why this is not a record.
@XmlRootElement(name = "product")
@XmlAccessorType(XmlAccessType.FIELD)
public class ProductDto {
    @XmlAttribute public long id;
    public String name;
    public String sku;
    public BigDecimal price;
    public int stock;

    public ProductDto() { }

    public static ProductDto from(Product p) {
        var d = new ProductDto();
        d.id = p.id(); d.name = p.name(); d.sku = p.sku(); d.price = p.price();
d.stock = p.stock();
        return d;
    }
```

```
    public Product toProduct(long id, int version) {
        return new Product(id, name, sku, price, stock, version);
    }
}


@POST
public Response create(ProductDto dto, @Context UriInfo uriInfo) {
    repo.findBySku(dto.sku).ifPresent(p -> { throw new
DuplicateSkuException(dto.sku); });
    Product saved = repo.save(dto.toProduct(0, 0));
    URI location =
uriInfo.getAbsolutePathBuilder().path(Long.toString(saved.id())).build();
    return Response.created(location).entity(ProductDto.from(saved)).build();
}

// TODO: PUT {id}, DELETE {id}, PATCH {id}/stock
(@Consumes(MediaType.APPLICATION_JSON))
```

EXPECTED OUTPUT

```
$ curl -i -X POST localhost:8080/api/products -H 'Content-Type:
application/json' \
       -d '{"name":"Teapot","sku":"TP-01","price":31.00,"stock":5}'
HTTP/1.1 201 Created
Location: http://localhost:8080/api/products/4
Content-Type: application/json

{"id":4,"name":"Teapot","price":31.00,"sku":"TP-01","stock":5}

$ curl -i -X POST localhost:8080/api/products -H 'Content-Type:
application/json' \
       -d '{"name":"Teapot","sku":"TP-01","price":31.00,"stock":5}'
HTTP/1.1 409 Conflict

$ curl -s localhost:8080/api/products/4 -H 'Accept: application/xml'
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<product id="4"><name>Teapot</name><sku>TP-01</sku><price>31.00</price>
<stock>5</stock></product>

$ curl -i -X PATCH localhost:8080/api/products/3/stock -H 'Content-Type:
application/json' -d '{"delta":-1}'
HTTP/1.1 409 Conflict

$ curl -i -X DELETE localhost:8080/api/products/4
HTTP/1.1 204 No Content
```

```
$ curl -i -X DELETE localhost:8080/api/products/4
HTTP/1.1 404 Not Found

$ curl -i -X POST localhost:8080/api/products -H 'Content-Type: text/plain' -d
'Teapot'
HTTP/1.1 415 Unsupported Media Type
```

Show hints

- `class DuplicateSkuException extends ClientErrorException { DuplicateSkuException(String sku) { super("SKU " + sku + " already exists", 409); } }`. Task 3 turns these into problem+json bodies; for now an empty 409 is fine.
- `repo.findById(id).orElseThrow(() -> new NotFoundException(...))` keeps the resource methods to two or three lines. Put the stock rule (`stock + delta < 0`) in the repository or a small `ProductService`, not in the resource.
- A lambda cannot throw from inside `ifPresent` unless the exception is unchecked; `ClientErrorException` is a `RuntimeException`, so it works.
- For XML input send `-H 'Content-Type: application/xml' -d '<product><name>Bowl</name><sku>BW-01</sku><price>9.90</price><stock>3</stock></product>'`.
- The 415 and the 404 for `/products/abc` cost you nothing: matching does them before your code runs (slide 14).

ACCEPTANCE CHECKLIST

- [ ] `POST` returns 201 with a `Location` header built from `UriInfo`; a duplicate SKU returns 409.
- [ ] `PUT` 200/404, `PATCH /stock` 200/409, `DELETE` 204/404 behave exactly as specified; no endpoint returns 200 for an error.
- [ ] Every representation is a `ProductDto`; JSON and XML both work for input and output, and `version` is not exposed.
- [ ] Business rules (duplicate SKU, negative stock) live outside the resource class; `docs/task2-http.md` shows all eight calls.

3 Parameters and validation **Medium** **60 min**

GOAL

Filter the product list through typed query parameters grouped in a `@BeanParam`, convert a custom `Money` type with a `ParamConverter`, validate DTOs and parameters with Bean Validation,

and turn every error into an RFC 9457 `application/problem+json` body.

STEPS

1. Create `domain/Money(BigDecimal amount, String currency)` with `static Money parse(String)` accepting `"12.50 EUR"` (throws `IllegalArgumentException` otherwise) and `toString()` producing the same format. Create `api/params/MoneyConverterProvider` (`@Provider`, `ParamConverterProvider`) that returns a converter only for `Money.class`.
2. Create `api/params/ProductFilter` (starter below) and change `GET /products` to `list(@BeanParam @Valid ProductFilter f)`. Filtering: `q` matches the name case-insensitively, `minPrice`/`maxPrice` compare the amount (currency must be `EUR`, otherwise 400), `inStock=true` keeps `stock > 0`. Sorting: `sort=field,dir` with the field whitelisted (`name`, `price`, `stock`); an unknown field is a 400.
3. Add constraints to `ProductDto`: `@NotBlank @Size(max = 80) name`, `@NotNull @Pattern(regexp = "[A-Z]{2}-\\d{2}") sku`, `@NotNull @DecimalMin("0.01") @Digits(integer = 8, fraction = 2) price`, `@PositiveOrZero stock`. Mark the body parameters of `POST` and `PUT` with `@Valid @NotNull`, and `StockChange.delta` with `@Min(-1000) @Max(1000)`.
4. Create `api/dto/Problem(String type, String title, int status, String detail, String instance, List<Map<String, String>> errors)` and, in `api/error`, two mappers: `ValidationMapper implements ExceptionMapper<ConstraintViolationException>` (400 with an `errors` list of field/message pairs) and `ProblemMapper implements ExceptionMapper<Throwable>` that reuses the status of a `WebApplicationException` (404, 409, 415…) and answers 500 with a logged reference id for anything else. Both set `Content-Type: application/problem+json` and `instance` to the request path.
5. Replace the plain 404/409 of Tasks 1 and 2 with problem bodies (no code change in the resource should be needed) and check that a 500 never shows a stack trace: temporarily throw an `IllegalStateException` from one method and look at the body.
6. Record `docs/task3-http.md`: a filtered and sorted list, `minPrice=abc` (400 from the converter), `sort=colour,asc` (400), an invalid POST with three field errors, a 404 as problem+json, and the 500 experiment.

STARTER CODE

```
package edu.itc.shop.api.params;

import jakarta.validation.constraints.Pattern;
import jakarta.ws.rs.DefaultValue;
import jakarta.ws.rs.QueryParam;
import edu.itc.shop.domain.Money;
```

```
public class ProductFilter {
    @QueryParam("q") public String q;
    @QueryParam("minPrice") public Money minPrice;                  // "5 EUR"
    @QueryParam("maxPrice") public Money maxPrice;
    @QueryParam("inStock") @DefaultValue("false") public boolean inStock;
    @QueryParam("sort") @DefaultValue("name,asc")
    @Pattern(regexp = "[a-z]+,(asc|desc)", message = "use field,asc or
field,desc")
    public String sort;

    public String sortField() { return sort.split(",")[0]; }
    public boolean descending() { return sort.endsWith(",desc"); }
}
```

```
package edu.itc.shop.api.error;

import jakarta.validation.ConstraintViolationException;
import jakarta.validation.Path;
import jakarta.ws.rs.core.*;
import jakarta.ws.rs.ext.*;
import edu.itc.shop.api.dto.Problem;

@Provider
public class ValidationMapper implements
ExceptionMapper<ConstraintViolationException> {
    @Context UriInfo uriInfo;

    @Override
    public Response toResponse(ConstraintViolationException ex) {
        var errors = ex.getConstraintViolations().stream()
                .map(v -> Map.of("field", lastNode(v.getPropertyPath()),
"message", v.getMessage()))
                .toList();
        var problem = new Problem("about:blank", "Invalid request", 400,
"Validation failed",
                                  uriInfo.getRequestUri().getPath(), errors);
        return
Response.status(400).type(Problem.MEDIA_TYPE).entity(problem).build();
    }

    // "create.arg0.price" → "price"
    static String lastNode(Path path) {
        String name = null;
        for (Path.Node n : path) name = n.getName();
        return name;
    }
}
```

```
// TODO: ProblemMapper implements ExceptionMapper<Throwable>
//   WebApplicationException → same status, title = reason phrase, detail =
message
//   anything else           → 500, log with System.getLogger("api"), detail =
"reference " + UUID
```

EXPECTED OUTPUT

```
$ curl -s 'localhost:8080/api/products?
minPrice=5%20EUR&inStock=true&sort=price,desc'
[{"id":1,"name":"Kettle","price":24.90,"sku":"KT-01","stock":12},
 {"id":2,"name":"Mug","price":6.50,"sku":"MG-01","stock":40}]

$ curl -i 'localhost:8080/api/products?minPrice=abc'
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{"detail":"Cannot parse Money from
'abc'","instance":"/api/products","status":400,
 "title":"Bad Request","type":"about:blank"}

$ curl -i -X POST localhost:8080/api/products -H 'Content-Type:
application/json' \
       -d '{"name":"","sku":"kettle","price":0,"stock":-1}'
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{"detail":"Validation failed",
 "errors":[{"field":"name","message":"must not be blank"},
           {"field":"sku","message":"must match \"[A-Z]{2}-\\d{2}\""},
           {"field":"price","message":"must be greater than or equal to 0.01"},
           {"field":"stock","message":"must be greater than or equal to 0"}],
 "instance":"/api/products","status":400,"title":"Invalid
request","type":"about:blank"}

$ curl -i localhost:8080/api/products/99
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{"detail":"Product 99 not found","instance":"/api/products/99","status":404,
 "title":"Not Found","type":"about:blank"}
```

Show hints

- Why two mappers? Jersey ships its own mapper for `ValidationException`; a mapper for `Throwable` is less specific and would lose. Your `ExceptionMapper<ConstraintViolationException>` is the most specific one and wins.
- A `ParamConverter.fromString` that throws `IllegalArgumentException` gives 400 for query parameters and 404 for path parameters, by specification. Throw `new BadRequestException("Cannot parse Money from '" + s + "'")` instead to keep your message.
- In the `Throwable` mapper: `if (ex instanceof WebApplicationException w) { int status = w.getResponse().getStatus(); String title = Response.Status.fromStatusCode(status).getReasonPhrase(); ... }`. Responses created with `abortWith` in filters bypass mappers, which is intended.
- Constraint messages come from Hibernate Validator's default bundle. If start-up complains about a missing EL implementation, add `org.glassfish.expressly:expressly:6.0.0`.
- Sorting: build a `Comparator<Product>` from a `Map<String, Comparator<Product>>` keyed by the whitelisted field names; `reversed()` for `desc`. An unknown key is `throw new BadRequestException("Unknown sort field " + field)`.

ACCEPTANCE CHECKLIST

- [ ] `GET /products` filters by `q`, `minPrice`/`maxPrice` (Money via `ParamConverter`), `inStock` and sorts by a whitelisted field; unknown fields and unparsable money give 400.
- [ ] Invalid `POST`/`PUT`/`PATCH` bodies return 400 problem+json with one entry per violated constraint.
- [ ] Every error (400, 404, 409, 415, 500) is `application/problem+json` with type, title, status, detail and instance; 500 bodies contain a reference id and no stack trace.
- [ ] Resource methods contain no `try/catch` and no validation code; `docs/task3-http.md` shows the six calls.

4 Providers and filters **Medium** **60 min**

GOAL

Add cross-cutting behaviour without touching the resource: a correlation id and access log, CORS headers, an API-key check that runs before matching, and gzip compression through a name-bound `WriterInterceptor`.

STEPS

1. `api/filter/CorrelationFilter` implements both `ContainerRequestFilter` and `ContainerResponseFilter` (`@Priority(Priorities.USER)`). Request side: take `X-Correlation-Id` from the request or generate a UUID, store it with `setProperty`.

Response side: add the header to the response and log one line `METHOD path -> status [id] ms` with `System.Logger`.

2. `api/filter/CorsFilter` (`ContainerResponseFilter`) adds `Access-Control-Allow-Origin: http://localhost:5173`, `Allow-Methods: GET,POST,PUT,PATCH,DELETE`, `Allow-Headers: Content-Type,X-Api-Key,X-Correlation-Id,If-Match`, `Expose-Headers: Location,ETag,Link,X-Total-Count,X-Correlation-Id`. A `@PreMatching` request filter answers `OPTIONS` with `abortWith(Response.ok().build())` so preflights never reach a resource.
3. `api/filter/ApiKeyFilter` (`@PreMatching`, `@Priority(Priorities.AUTHENTICATION)`): for every method except `GET`, `HEAD` and `OPTIONS` require header `X-Api-Key` with a value from the system property `shop.api.keys` (comma-separated, default `demo-123`). Otherwise `abortWith` a 401 *problem+json* body and `WWW-Authenticate: ApiKey realm="shop"`.
4. Create the `@NameBinding` annotation `api/filter/Compress` and `GzipWriterInterceptor` (`@Provider @Compress`). Inject `@Context HttpHeaders` to read the request's `Accept-Encoding`; only when it contains `gzip` wrap the output stream and set `Content-Encoding: gzip` and `Vary: Accept-Encoding`. Annotate the list method with `@Compress`.
5. Check the order with logging: add a temporary `System.out.println(getClass().getSimpleName())` in each request filter and confirm `ApiKeyFilter` runs before `CorrelationFilter`; then remove the prints.
6. Record `docs/task4-http.md`: a POST without key (401 problem+json), with a wrong key (401), with the right key (201), an `OPTIONS` preflight, a GET showing `X-Correlation-Id` echoed back, `curl --compressed -v` showing `Content-Encoding: gzip`, and the same GET without `Accept-Encoding` showing no compression.

STARTER CODE

```
package edu.itc.shop.api.filter;

import java.util.Set;
import jakarta.annotation.Priority;
import jakarta.ws.rs.Priorities;
import jakarta.ws.rs.container.*;
import jakarta.ws.rs.core.Response;
import jakarta.ws.rs.ext.Provider;
import edu.itc.shop.api.dto.Problem;

@Provider
@PreMatching
@Priority(Priorities.AUTHENTICATION)
public class ApiKeyFilter implements ContainerRequestFilter {
    private static final Set<String> SAFE = Set.of("GET", "HEAD", "OPTIONS");
    private final Set<String> keys =
```

```
            Set.of(System.getProperty("shop.api.keys", "demo-123").split(","));

    @Override
    public void filter(ContainerRequestContext ctx) {
        if (SAFE.contains(ctx.getMethod())) return;
        String key = ctx.getHeaderString("X-Api-Key");
        if (key != null && keys.contains(key)) return;
        var problem = new Problem("about:blank", "Unauthorized", 401,
                "Missing or invalid X-Api-Key",
ctx.getUriInfo().getRequestUri().getPath(), null);
        ctx.abortWith(Response.status(401)
                .header("WWW-Authenticate", "ApiKey realm=\"shop\"")
                .type(Problem.MEDIA_TYPE).entity(problem).build());
    }
}


@Provider
@Compress
public class GzipWriterInterceptor implements WriterInterceptor {
    @Context HttpHeaders requestHeaders;      // providers may use @Context
injection

    @Override
    public void aroundWriteTo(WriterInterceptorContext ctx) throws IOException {
        String enc =
requestHeaders.getHeaderString(HttpHeaders.ACCEPT_ENCODING);
        if (enc != null && enc.contains("gzip")) {
            // TODO: set Content-Encoding and Vary, wrap ctx.getOutputStream()
in a GZIPOutputStream
        }
        ctx.proceed();
    }
}
```

EXPECTED OUTPUT

```
$ curl -i -X POST localhost:8080/api/products -H 'Content-Type:
application/json' -d '{...}'
HTTP/1.1 401 Unauthorized
WWW-Authenticate: ApiKey realm="shop"
Content-Type: application/problem+json
X-Correlation-Id: 3f1c0d5e-...
Access-Control-Allow-Origin: http://localhost:5173

{"detail":"Missing or invalid X-Api-
Key","instance":"/api/products","status":401,
```

```
 "title":"Unauthorized","type":"about:blank"}

$ curl -i -X OPTIONS localhost:8080/api/products -H 'Origin:
http://localhost:5173' \
       -H 'Access-Control-Request-Method: POST'
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Methods: GET,POST,PUT,PATCH,DELETE
Access-Control-Allow-Headers: Content-Type,X-Api-Key,X-Correlation-Id,If-Match

$ curl -si localhost:8080/api/products/1 -H 'X-Correlation-Id: order-check-42' |
grep -i correlation
X-Correlation-Id: order-check-42

$ curl -sv --compressed localhost:8080/api/products -o /dev/null 2>&1 | grep -i
-E 'encoding|vary'
> Accept-Encoding: deflate, gzip, br, zstd
< Content-Encoding: gzip
< Vary: Accept-Encoding

server log:
INFO: POST /api/products -> 401 [3f1c0d5e-...] 2 ms
INFO: GET  /api/products/1 -> 200 [order-check-42] 1 ms
```

Show hints

- Timing: store `System.nanoTime()` as a request property in the request filter and read it back in the response filter of the same class.
- `abortWith` responses skip the exception mappers but not the response filters, so the 401 still carries CORS and correlation headers. That is why the 401 body must be built by the filter itself.
- Name binding needs `@Retention(RUNTIME)` and `@Target({TYPE, METHOD})` on `@Compress`, and the annotation on both the provider class and the resource method.
- Remove `Content-Length` when compressing: `ctx.getHeaders().remove(HttpHeaders.CONTENT_LENGTH)`; Grizzly then uses chunked transfer.
- Pass keys at start-up with `mvn -q compile exec:java -Dexec.args= -Dshop.api.keys=demo-123,ci-777`; never commit real keys.

ACCEPTANCE CHECKLIST

- [ ] Every response carries `X-Correlation-Id` (echoed or generated) and one access-log line per request is printed with method, path, status, id and duration.
- [ ] CORS headers are present on all responses and `OPTIONS` preflights return 200 without reaching a resource.

- [ ] Writes without a valid `X-Api-Key` are rejected with 401 problem+json and `WWW-Authenticate` before matching; reads stay public.
- [ ] `GET /products` is gzipped only when the client accepts gzip (`Content-Encoding`, `Vary`), other methods are untouched; `docs/task4-http.md` is complete.

5 Pagination, caching and versioning

Hard — 75 min

GOAL

Make the list scalable and the item cacheable: page/size with `Link` headers and a total count, `ETag` with 304 and 412, `Cache-Control`, a v2 representation selected by media type, and a `/products/{id}/reviews` sub-resource.

STEPS

1. Add `@QueryParam("page") @DefaultValue("0") @Min(0) int page` and `@QueryParam("size") @DefaultValue("20") @Min(1) @Max(100) int size` to `ProductFilter`. Seed 30 more products in the repository (a loop is fine) so paging is visible. `GET /products` now returns a `Response`: the page as body, `X-Total-Count` (after filtering) and `Link` headers `first`, `prev`, `next`, `last` built with `UriInfo.getRequestUriBuilder().replaceQueryParam("page", n)`. Omit `prev` on the first and `next` on the last page.
2. `GET /products/{id}`: compute `EntityTag` from `version`, call `request.evaluatePreconditions(tag)` and return its builder (304) when non-null; otherwise add `tag(...)` and `cacheControl(...)` with `private, max-age=60, no-transform`.
3. `PUT /products/{id}`: require `If-Match` (428 problem+json when missing), answer 412 when it does not match the current version, otherwise save and return the new `ETag`. `PATCH /stock` keeps working without preconditions.
4. Versioning by media type: create the record `api/dto/ProductV2(long id, String name, String sku, Money price, int stock, boolean available)`. Add a second `GET {id}` method with `@Produces("application/vnd.shop.v2+json")`; mark the existing JSON as `"application/json; qs=1.0"` and `"application/vnd.shop.v1+json"` so plain clients keep v1.
5. Sub-resource: `domain/Review(long id, long productId, int stars, String author, String text)` in a `ReviewRepository` singleton (bind it in `Main.config()`). The locator `@Path("{id}/reviews")` in `ProductResource` returns `new ReviewResource(product, reviewRepo)`; `ReviewResource` offers `GET` (list) and `POST` (`@Valid` body with `@Min(1)`

`@Max(5) stars`, `@NotBlank author`, `@Size(max = 500) text`; 201 + `Location`). Reviews for an unknown product are 404 because the locator already fails.

6. Record `docs/task5-http.md`: page 1 of size 10 with headers, a GET with `ETag`, the 304, a PUT without `If-Match` (428), with a stale tag (412), with the right tag (200 + new ETag), v1 and v2 of the same product, POST and GET of a review, and reviews of product 999 (404).

STARTER CODE

```
@GET
@Compress
public Response list(@BeanParam @Valid ProductFilter f, @Context UriInfo
uriInfo) {
    List<Product> all = service.search(f);                      // filtered +
sorted
    int total = all.size();
    int lastPage = Math.max(0, (total - 1) / f.size);
    List<ProductDto> page = all.stream().skip((long) f.page *
f.size).limit(f.size)
                               .map(ProductDto::from).toList();

    var links = new ArrayList<Link>();
    links.add(link(uriInfo, 0, "first"));
    // TODO: prev (page > 0), next (page < lastPage), last
    return Response.ok(new GenericEntity<List<ProductDto>>(page) {})
                   .links(links.toArray(Link[]::new))
                   .header("X-Total-Count", total)
                   .build();
}

private static Link link(UriInfo uriInfo, int page, String rel) {
    return
Link.fromUriBuilder(uriInfo.getRequestUriBuilder().replaceQueryParam("page",
page))
               .rel(rel).build();
}

@GET @Path("{id: \\d+}")
@Produces({"application/json; qs=1.0", "application/vnd.shop.v1+json",
MediaType.APPLICATION_XML})
public Response one(@PathParam("id") long id, @Context Request request) {
    Product p = service.get(id);                                 // 404 inside
    EntityTag tag = new EntityTag(Integer.toHexString(p.version()));
    Response.ResponseBuilder cond = request.evaluatePreconditions(tag);
    if (cond != null) return cond.tag(tag).build();             // 304
    CacheControl cc = new CacheControl();
    cc.setPrivate(true); cc.setMaxAge(60); cc.setNoTransform(true);
    return Response.ok(ProductDto.from(p)).tag(tag).cacheControl(cc).build();
```

```
}

// TODO: @GET @Path("{id: \\d+}") @Produces("application/vnd.shop.v2+json") →
ProductV2
// TODO: PUT with @HeaderParam(HttpHeaders.IF_MATCH) String ifMatch → 428 / 412
/ 200
```

EXPECTED OUTPUT

```
$ curl -si 'localhost:8080/api/products?page=1&size=10' | grep -i -E
'link|total|^HTTP'
HTTP/1.1 200 OK
X-Total-Count: 33
Link: <http://localhost:8080/api/products?page=0&size=10>; rel="first",
      <http://localhost:8080/api/products?page=0&size=10>; rel="prev",
      <http://localhost:8080/api/products?page=2&size=10>; rel="next",
      <http://localhost:8080/api/products?page=3&size=10>; rel="last"

$ curl -si localhost:8080/api/products/1 | grep -i -E 'etag|cache|^HTTP'
HTTP/1.1 200 OK
ETag: "1"
Cache-Control: private, no-transform, max-age=60

$ curl -si localhost:8080/api/products/1 -H 'If-None-Match: "1"'
HTTP/1.1 304 Not Modified
ETag: "1"

$ curl -si -X PUT localhost:8080/api/products/1 -H 'X-Api-Key: demo-123' \
       -H 'Content-Type: application/json' -d '{"name":"Kettle XL","sku":"KT-
01","price":29.90,"stock":12}'
HTTP/1.1 428 Precondition Required

$ curl -si -X PUT localhost:8080/api/products/1 -H 'X-Api-Key: demo-123' -H 'If-
Match: "0"' \
       -H 'Content-Type: application/json' -d '{...}'
HTTP/1.1 412 Precondition Failed

$ curl -si -X PUT localhost:8080/api/products/1 -H 'X-Api-Key: demo-123' -H 'If-
Match: "1"' \
       -H 'Content-Type: application/json' -d '{...}' | grep -i -E 'etag|^HTTP'
HTTP/1.1 200 OK
ETag: "2"

$ curl -s localhost:8080/api/products/1 -H 'Accept:
application/vnd.shop.v2+json'
{"available":true,"id":1,"name":"Kettle XL","price":
{"amount":29.90,"currency":"EUR"},"sku":"KT-01","stock":12}
```

```
$ curl -si -X POST localhost:8080/api/products/1/reviews -H 'X-Api-Key: demo-
123' \
       -H 'Content-Type: application/json' -d
'{"stars":5,"author":"Ana","text":"Boils fast."}' | grep -E '^HTTP|Location'
HTTP/1.1 201 Created
Location: http://localhost:8080/api/products/1/reviews/1

$ curl -si localhost:8080/api/products/999/reviews | head -1
HTTP/1.1 404 Not Found
```

Show hints

- Two methods with the same `@Path` and different `@Produces` are legal: the media type decides. If both could match (client sends `Accept: */*`), the `qs` value breaks the tie in favour of v1.
- `evaluatePreconditions` handles `If-None-Match` (GET → 304) and `If-Match` (PUT → 412) alike; the 428 for a missing `If-Match` is your own check with `@HeaderParam`.
- `ETag` values are quoted strings: `new EntityTag("1")` renders as `"1"`. Clients must send the quotes back; `-H 'If-Match: "1"'` in curl.
- Keep the sub-resource locator free of HTTP method annotations; the `ReviewResource` instance is created per request and therefore may hold the `Product` as a field.
- Money in JSON: JSON-B serialises the record as an object; if you want `"amount":"29.90"` as a string, add a `JsonbSerializer` or make `amount` a `String` in `ProductV2`.

ACCEPTANCE CHECKLIST

- [ ] `page`/`size` are validated, `X-Total-Count` reflects the filtered total and the `Link` header carries the correct `first`/`prev`/`next`/`last` relations built from the request URI.
- [ ] `GET /products/{id}` returns `ETag` and `Cache-Control`, answers 304 to a matching `If-None-Match`; `PUT` answers 428 without `If-Match`, 412 when stale, 200 with the new tag otherwise.
- [ ] `Accept: application/vnd.shop.v2+json` returns the v2 shape (Money object, `available`); plain JSON clients still get v1.
- [ ] `/products/{id}/reviews` is served by a sub-resource locator with validated `POST` (201 + Location) and 404 for unknown products; `docs/task5-http.md` shows all ten calls.

★

Challenge: orders with HATEOAS, idempotent POST, OpenAPI and a full test suite

Hard **90–120 min · optional, extra credit**

SCENARIO

The shop front-end needs to place orders. A mobile client on a bad connection retries a POST after a timeout, so placing an order must be idempotent. The team wants the API documented as OpenAPI and every promised status code covered by a test that runs in CI with `mvn verify`. Finally, a colleague coming from Spring asks how this compares to Chapter 02.

REQUIREMENTS

1. **Orders sub-API.** `POST /orders` with body `PlaceOrder(List<Line(long productId, @Min(1) int quantity)>)` reserves stock (409 when insufficient, 404 for an unknown product), creates an `Order(String id, List<Line>, BigDecimal total, OrderStatus status)` with status `NEW` and answers 201 + Location. `GET /orders/{id}`, `PUT /orders/{id}/payment` (NEW → PAID, 409 otherwise) and `DELETE /orders/{id}` (NEW → CANCELLED, restores stock; 409 when PAID).
2. **HATEOAS.** Every order representation carries `Link` headers and a `_links` map with `self` always, `pay` and `cancel` only while `NEW`, and `product` links for each line. A client must be able to drive the whole flow following `rel`s only.
3. **Idempotency.** `POST /orders` requires `Idempotency-Key` (428 when missing). Store the first response per key in a `ConcurrentHashMap` (bound singleton) together with a SHA-256 of the body; replay it with `Idempotency-Replayed: true` for the same key and body, answer 422 for the same key with a different body. Use `computeIfAbsent` so two concurrent retries create one order.
4. **OpenAPI.** Add `org.eclipse.microprofile.openapi:microprofile-openapi-api:4.1` (scope `provided`) and the `io.smallrye:smallrye-open-api-maven-plugin` bound to `process-classes`, writing `openapi.yaml` into `target/classes/META-INF`. Annotate at least the product and order resources with `@Tag`, `@Operation`, `@APIResponse` for every status code they can return, and `@Parameter`. Serve the file at `GET /openapi` with `Content-Type: application/yaml` from a small `OpenApiResource`.
5. **Tests.** `ShopApiTest extends JerseyTest` with `configure()` returning `Main.config()` plus a fresh repository per class. One test per status code: 200, 201, 204, 304, 400, 401, 404, 406, 409, 412, 415, 422, 428. Assert status, `Content-Type` (problem+json on errors) and the important header (`Location`, `ETag`, `Link`, `Idempotency-Replayed`). `mvn -q verify` must be green.
6. **README.** Add a section "JAX-RS vs Spring MVC" with the `GET /products/{id}` and `POST /products` endpoints written both ways (Jakarta REST from this lab, Spring MVC from Lab-

02) and a table of eight differences (annotations, body binding, response building, errors, filters, JSON library, DI, testing).

SKELETON

```
@Path("/orders")
@Produces(MediaType.APPLICATION_JSON)
public class OrderResource {
    private record Stored(String bodyHash, Response.StatusType status, URI
location, Object body) {}

    private final OrderService orders;
    private final ConcurrentMap<String, Stored> idempotency;    // bound
singleton

    @Inject
    public OrderResource(OrderService orders, ConcurrentMap<String, Stored>
idempotency) { ... }

    @POST
    @Consumes(MediaType.APPLICATION_JSON)
    public Response place(@HeaderParam("Idempotency-Key") String key,
                          @Valid @NotNull PlaceOrder cmd, @Context UriInfo
uriInfo) {
        if (key == null || key.isBlank())
            throw new ClientErrorException("Idempotency-Key header required",
428);
        String hash = Hashes.sha256(cmd);
        boolean[] replayed = { true };
        Stored s = idempotency.computeIfAbsent(key, k -> {
            replayed[0] = false;
            Order order = orders.place(cmd);                        // 404 / 409
inside
            URI loc = uriInfo.getAbsolutePathBuilder().path(order.id()).build();
            return new Stored(hash, Response.Status.CREATED, loc,
represent(order, uriInfo));
        });
        if (!s.bodyHash().equals(hash))
            throw new ClientErrorException("Idempotency-Key reused with a
different body", 422);
        return Response.status(s.status()).location(s.location())
                       .header("Idempotency-Replayed", replayed[0])
                       .entity(s.body()).build();
    }

    // TODO: GET {id}, PUT {id}/payment, DELETE {id}
    // TODO: represent(order, uriInfo) → OrderDto with _links (self, pay,
```

```
cancel, product…)
}


class ShopApiTest extends JerseyTest {
    @Override
    protected Application configure() {
        return Main.config();                                    // same app as
production
    }

    @Test
    void placingTheSameOrderTwiceCreatesOneOrder() {
        var body = Entity.json("""
                {"lines":[{"productId":1,"quantity":2}]}
                """);
        Response first = target("orders").request().header("X-Api-Key", "demo-
123")
                .header("Idempotency-Key", "k-1").post(body);
        Response second = target("orders").request().header("X-Api-Key", "demo-
123")
                .header("Idempotency-Key", "k-1").post(body);
        assertEquals(201, first.getStatus());
        assertEquals(201, second.getStatus());
        assertEquals("true", second.getHeaderString("Idempotency-Replayed"));
        assertEquals(first.getLocation(), second.getLocation());
    }

    // TODO: one test per status code: 200, 204, 304, 400, 401, 404, 406, 409,
412, 415, 422, 428
}
```

EXPECTED OUTPUT

```
$ curl -si -X POST localhost:8080/api/orders -H 'X-Api-Key: demo-123' -H
'Idempotency-Key: 7c1e' \
       -H 'Content-Type: application/json' -d '{"lines":
[{"productId":1,"quantity":2}]}'
HTTP/1.1 201 Created
Location: http://localhost:8080/api/orders/o-1001
Link: <http://localhost:8080/api/orders/o-1001>; rel="self",
      <http://localhost:8080/api/orders/o-1001/payment>; rel="pay",
      <http://localhost:8080/api/orders/o-1001>; rel="cancel"
Idempotency-Replayed: false

{"id":"o-1001","status":"NEW","total":49.80,
 "lines":[{"productId":1,"quantity":2}],
```

```
 "_links":{"self":"http://localhost:8080/api/orders/o-1001",
           "pay":"http://localhost:8080/api/orders/o-1001/payment",
           "cancel":"http://localhost:8080/api/orders/o-1001",
           "product-1":"http://localhost:8080/api/products/1"}}

$ (same command again)
HTTP/1.1 201 Created
Idempotency-Replayed: true

$ curl -si -X POST localhost:8080/api/orders -H 'X-Api-Key: demo-123' -H
'Idempotency-Key: 7c1e' \
       -H 'Content-Type: application/json' -d '{"lines":
[{"productId":2,"quantity":1}]}' | head -1
HTTP/1.1 422 Unprocessable Content

$ curl -s localhost:8080/api/openapi | head -6
openapi: 3.1.0
info:
  title: shop-rest API
  version: 1.0.0
paths:
  /orders:

$ mvn -q verify
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0
```

Show hints

- Bind the idempotency map as `bind(new ConcurrentHashMap<String, Object>()).to(new TypeLiteral<ConcurrentMap<String, Object>>() {})`, or wrap it in a small `IdempotencyStore` class, which is easier to inject and to test.
- Hash the JSON body, not the Java object: inject `Jsonb` or serialise `cmd` with `JsonbBuilder.create().toJson(cmd)` before `MessageDigest.getInstance("SHA-256")`.
- The SmallRye plugin configuration: `<goals><goal>generate-schema</goal></goals>`, `<outputDirectory>${project.build.outputDirectory}/META-INF</outputDirectory>`, `<infoTitle>shop-rest API</infoTitle>`, `<infoVersion>1.0.0</infoVersion>`. Serve it with `getClass().getResourceAsStream("/META-INF/openapi.yaml")`.
- `JerseyTest` starts one Grizzly per test class on port 9998 by default; set `System.setProperty("jersey.config.test.container.port", "0")` in a static block for a random port. Tests share the repository within the class, so read ids from `Location` instead of hard-coding them.
- For 406 send `request("text/csv")`; for 415 post `Entity.text("x")`; for 304 first GET the `ETag`, then `header("If-None-Match", etag)`.

ACCEPTANCE CHECKLIST

- [ ] Orders can be placed, paid and cancelled following only the `rel` links; stock is reserved and restored; invalid transitions give 409.
- [ ] Replaying a POST with the same `Idempotency-Key` returns the stored 201 with `Idempotency-Replayed: true`; a different body gives 422, a missing key 428; concurrent retries create one order.
- [ ] `GET /api/openapi` serves a generated OpenAPI 3.1 document that lists every endpoint and every status code the tests cover.
- [ ] `mvn -q verify` runs a `JerseyTest` suite with at least 13 status-code tests, all green; the README compares the same endpoints in JAX-RS and Spring MVC.

Grading rubric

Component	Points	Full marks when…
Task 1 · First resource and server	10	Grizzly starts from `Main`, the repository is a bound singleton injected by constructor, list and item endpoints work, unknown ids are 404.
Task 2 · CRUD, status codes, JSON and XML	15	All six endpoints return the specified codes and headers (201 + Location, 204, 404, 409, 415); DTOs serve JSON and XML in both directions; rules live outside the resource.
Task 3 · Parameters and validation	20	`@BeanParam` filter with `ParamConverter`, whitelisted sorting, Bean Validation on bodies and parameters, every error an RFC 9457 problem+json with correct status; 500 leaks nothing.
Task 4 · Providers and filters	20	Correlation id + access log, CORS + preflight, pre-matching API-key filter with priority, name-bound gzip interceptor honouring `Accept-Encoding`; order of filters demonstrated.
Task 5 · Pagination, caching, versioning	25	Correct `Link` relations and total count, ETag/304/412/428 semantics, `Cache-Control`, v2 by vendor media type with v1 default, validated sub-resource with 404 through the locator.
Code quality and documentation	10	Thin stateless resources, no `try/catch` in resources, meaningful names, `docs/task*-http.md` complete and readable, README explains how to run and test.
★ Challenge (extra credit)	+20	HATEOAS orders flow, race-free idempotency with body hash, generated OpenAPI served at `/openapi`, 13+ green

Component	Points	Full marks when…
		`JerseyTest`s, JAX-RS vs Spring MVC comparison.
Total	**100 (+20)**	

Automatic deductions: −5 for each 2xx returned for an error condition, or each error without an `application/problem+json` body; −5 for a hard-coded `http://localhost:8080` in a `Location` or `Link` header instead of `UriInfo`; −10 if a resource class contains business rules or `try/catch`; −5 for mutable per-request state in resource fields or a non-thread-safe shared collection in a singleton; −5 for a sort or filter field forwarded unchecked; −10 for committed API keys or `target/`.

Submission

1. Repository layout:

```
shop-rest/
├── README.md                       # how to run, how to test, JAX-RS vs
Spring MVC (challenge)
├── pom.xml
├── .gitignore                      # target/, .idea/, *.iml
├── docs/
│   ├── task1-http.md … task5-http.md
│   └── challenge-http.md           # challenge only
└── src/
    ├── main/java/edu/itc/shop/…
    └── test/java/edu/itc/shop/ShopApiTest.java   # challenge only
```

2. Verification from a clean checkout: `mvn -q verify` must pass (with the challenge it runs the test suite), then `mvn -q compile exec:java` must start the server and the transcripts in `docs/` must be reproducible against it.
3. Branch `lab-04`, merge request titled `Lab-04 – <your name>`. Mention in the description which tasks and which parts of the challenge you completed.

Back to Chapter 04 slides | Retake the chapter quiz | ← Lab-03